

Technical Specification: Dynamic Scoring & Percentile Benchmarking

This document outlines the requirements for implementing configurable scoring features and upper-market percentile benchmarking within the ChianParser application. All implementations must adhere to the project's core design patterns, utilizing SwiftUI for the presentation layer and SwiftData for persistent storage, following the strict MVVM architecture pattern with clean Dependency Injection layers.

1. Data Model Extension (SwiftData)

To support user-controlled feature toggles and persistent scoring configurations, a new persistent model must be introduced. This model will manage the states of individual scoring sub-systems and custom thresholds.

```
import Foundation
import SwiftData

@Model
class ScoringConfiguration {
    var id: UUID = UUID()

    // Feature Toggles
    var isCustomAreaScoreEnabled: Bool = true
    var isPercentileBenchmarkEnabled: Bool = true

    // Global Filtering Rules
    var excludeStudios: Bool = true
    var excludeApartments: Bool = true

    // Benchmarking Parameters
    var targetPercentile: Double = 0.80 // Default to 80th
    percentile (Upper Market)

    // Configurable Score Weights
    var priceScoreWeight: Int = 40
    var metroProximityWeight: Int = 25
    var locationFloorWeight: Int = 20
    var areaScoreWeight: Int = 15

    init() {}
}
```

2. Advanced Property Filtering Strategy

Filtering out unwanted property types (studios and apartments) must happen early in the pipeline to prevent pollution of the local SwiftData store and ensure accurate benchmark metrics.

Property Attribute	Action upon Match	Architectural Layer
isStudio == true	Immediate discard / Skip injection	CianDetailParser Pipeline
isApartment == true	Immediate discard / Skip injection	CianDetailParser Pipeline

3. Dynamic Area Evaluation Matrix

When the dynamic Area Score feature toggle is active, the evaluation system switches from a linear scale to a configurable distribution matrix optimized for flippability and asset liquidity.

```
struct AreaEvaluationMatrix {
    static func calculateScore(for area: Double, config:
ScoringConfiguration) -> Int {
        guard config.isCustomAreaScoreEnabled else {
            // Fallback to legacy linear calculation if disabled
            if area >= 60 { return 15 }
            if area >= 45 { return 11 }
            if area >= 30 { return 6 }
            return 2
        }

        // Liquidity-optimized Bell Curve
        switch area {
        case 35.0...50.0:
            return config.areaScoreWeight // Perfect 1-2 room target
allocations
        case 25.0...34.9:
            return Int(Double(config.areaScoreWeight) * 0.73) //
High liquidity micro-units
        case 51.0...70.0:
            return Int(Double(config.areaScoreWeight) * 0.40) //
Medium liquidity family spaces
        default:
```

```

        return 2 // Low flippability overhead profiles (<25m² or
>70m²)
    }
}

```

4. Robust Upper-Market Percentile Benchmarking

To measure the true arbitrage potential against target renovated exit prices instead of the depressed local median baseline, the scoring analyzer must implement a secure statistical percentile selector.

```

struct MarketBenchmarkAnalyzer {
    static func calculateTargetExitPrice(from apartments:
[Apartment], config: ScoringConfiguration) -> Double? {
        guard !apartments.isEmpty else { return nil }

        let sortedPricesSqm = apartments.map { $0.priceSqm
}.sorted()

        // Toggle routing logic
        let targetPercentile = config.isPercentileBenchmarkEnabled ?
config.targetPercentile : 0.50

        let targetIndex = Int(Double(sortedPricesSqm.count) *
targetPercentile)
        let safeIndex = max(0, min(targetIndex,
sortedPricesSqm.count - 1))

        return sortedPricesSqm[safeIndex]
    }
}

```

5. SwiftUI Configuration Interface (macOS Settings Control)

The user controls must be accessible through a dedicated configuration pane integrated directly inside the standard macOS Settings panel pattern.

```

struct ScoringSettingsView: View {
    @Bindable var config: ScoringConfiguration

    var body: some View {

```

```

Form {
    Section(header: Text("Feature Flags").font(.headline)) {
        Toggle("Enable Liquidity-Optimized Area Evaluation",
isOn: $config.isCustomAreaScoreEnabled)
        Toggle("Use Upper-Market Percentile Target", isOn:
$config.isPercentileBenchmarkEnabled)
    }

    if config.isPercentileBenchmarkEnabled {
        Section(header: Text("Market Benchmarking
Target").font(.headline)) {
            Slider(value: $config.targetPercentile, in:
0.50...0.95, step: 0.05) {
                Text("Target Percentile Baseline")
            } minimumValueLabel: {
                Text("50th (Median)")
            } maximumValueLabel: {
                Text("95th (Premium)")
            }
            Text("Current Focus:
\((Int(config.targetPercentile * 100))th percentile value.")
                .font(.caption)
                .foregroundColor(.secondary)
        }
    }

    Section(header: Text("Global
Exclusions").font(.headline)) {
        Toggle("Auto-Drop Studio Units", isOn:
$config.excludeStudios)
        Toggle("Auto-Drop Commercial Apartments", isOn:
$config.excludeApartments)
    }
    .padding()
    .frame(width: 480, height: 350)
}
}

```